PLANIFICACIÓN AUTOMÁTICA

PRÁCTICA 1

PLANIFICACIÓN CLÁSICA CON PDDL

22/03/2026

JAVIER ANDREI DUTU
JAIME CRIADO NIETO
KONSTANTIN YANEV MIHOV

Tabla de Contenidos

1
PARTE 1. PLANIFICACIÓN CLÁSICA CON PDDL 4
1.1. Modelado del dominio 5
1.1.1. Descripción general del problema 5
1.1.2. Tipos del dominio 5
1.1.3. Predicados 5
1.1.4. Acciones del dominio 6
1.1.5. Justificación del modelado 7
1.1.6. Problemas de prueba 7
1.2. Generación automática de problemas y benchmark con FF 8
1.2.1. Generador de problemas 8
1.2.2. Metodología experimental con FF 8
1.2.3. Resultados del benchmark con FF 9
1.2.4. Discusión de los resultados de FF 10
1.3. Comparativa con Pyperplan 11
1.3.1. Comparación entre BFS, IDS, A* y GBFS 11
1.3.2. Comparación entre GBFS y EHC con distintas heurísticas 12
1.3.3. Comparación entre BFS, IDS y A* con heurísticas admisibles 13
1.4. Conclusiones de la Parte 1 15
1.5. Resumen final de resultados 16
PARTE 2.1. AMPLIACIÓN DEL DOMINIO CON TRANSPORTADORES 17
2.1.1. Objetivo de la ampliación 18
2.1.2. Cambios respecto al dominio de la Parte 1 18
2.1.3. Tipos y predicados del nuevo dominio 19
2.1.4. Acciones del dominio 20
2.1.5. Justificación del modelado 22
2.1.6. Adaptación de los problemas experimentales 23
2.1.7. Repetición del experimento equivalente a 1.3.2 23
2.1.8. Repetición del experimento equivalente a 1.3.3 25
2.1.9. Discusión global de resultados 26
2.1.10. Conclusiones de la Parte 2.1 26
2.1.11. Resumen final de resultados 27
PARTE 2.2. PLANIFICACIÓN CON COSTES Y OPTIMIZACIÓN 28
2.2.1. Objetivo de la ampliación 29
2.2.2. Cambios en el dominio 29
2.2.3. Acciones y política de costes 30
2.2.4. Generación de problemas con costes 31
2.2.5. Metodología experimental 31
2.2.6. Planificadores satisficing evaluados 32
2.2.7. Planificadores óptimos evaluados 34
2.2.8. Comparación entre satisficing y óptimos 35
2.2.9. Interpretación de los resultados 36
2.2.10. Conclusiones de la Parte 2.2 36
2.2.11. Resumen final de resultados 37
3. Parte 3: Planificación con tiempos 38
3.1. Objetivo de la parte 39
3.2. Adaptación del dominio a planificación temporal 39
3.2.1. Paso a acciones durativas 39
3.2.2. Política de duraciones 40
3.2.3. Restricciones de concurrencia 40
3.2.4. Comentario sobre el modelado 41
3.3. Adaptación del generador de problemas 41

3.4. Validación inicial 41
3.5. Metodología experimental 43
3.5.1. Planificador utilizado y criterio de éxito 43
3.5.2. Configuraciones estudiadas 43
3.5.3. Localización del umbral de resolución 43
3.6. Resultados 44
3.6.1. Resumen por configuración 44
3.6.2. Comparación entre primera y última solución 45
3.7. Interpretación de resultados 48
3.7.1. Diferencia entre primera y última solución 48
3.7.2. Evolución del makespan y del número de pasos 48
3.7.3. Papel de la concurrencia 48
3.7.4. Papel del transportador 49
3.7.5. Por qué más recursos no siempre ayudan a OPTIC 49
3.7.6. Sobre el uso del timeout 49
3.8. Conclusiones 50
ANEXO I. Comandos a utilizar 51
1.1. Ejecución del dominio y los problemas 51
1.2. Generación automática de problemas y benchmark con FF 51
1.3. Comparativa con Pyperplan 51
2.1. Ejecución del dominio con transportadores 53
2.2. Ejecución del dominio con costes y optimización 53
2.3. Limpieza y repetición de experimentos 54
3.1. Ejecución manual del dominio temporal con OPTIC 55
3.2. Generación automática de problemas temporales 55
3.3. Benchmark temporal con OPTIC 55
3.4. Limpieza y repetición de experimentos 55

PARTE 1. PLANIFICACIÓN CLÁSICA CON PDDL

1.1. Modelado del dominio

1.1.1. Descripción general del problema

En esta primera parte se ha modelado un dominio de planificación clásica en PDDL para representar un servicio de reparto de suministros mediante drones. La idea del problema es sencilla: un dron parte desde un depósito en el que hay varias cajas, cada caja contiene un determinado tipo de contenido y varias personas distribuidas por distintas localizaciones necesitan recibir uno o varios de esos contenidos. El objetivo consiste en construir un plan que permita satisfacer las necesidades indicadas en la meta.

El modelado se ha realizado en PDDL clásico, utilizando únicamente :strips y :typing, tal como exige el enunciado. Se ha evitado el uso de extensiones adicionales y se ha optado por una representación compacta y cercana al dominio clásico Gripper, pero adaptada al contexto de drones y cajas. La decisión de basarse en una estructura parecida a Gripper resulta natural porque el dron dispone de dos brazos, puede transportar simultáneamente hasta dos cajas y debe desplazarse entre localizaciones para realizar entregas.

La principal decisión de diseño ha sido representar el contenido de las cajas de forma genérica. En lugar de crear predicados específicos para cada tipo de recurso, se ha definido un tipo content y un predicado que relaciona cada caja con el contenido que transporta. De este modo, el dominio es reutilizable y no queda restringido a un conjunto cerrado de recursos. También se ha representado explícitamente el hecho de que una persona ya haya recibido un determinado contenido mediante un predicado específico, lo que permite formular metas de forma directa y clara.

1.1.2. Tipos del dominio

El dominio define los siguientes tipos:

- location: localizaciones del problema, incluido el depósito.
- person: personas que pueden necesitar contenidos.
- drone: drones encargados del reparto.
- crate: cajas transportadas por el dron.
- content: contenido transportado por las cajas.
- arm: brazos del dron.

Aunque en los experimentos de esta primera parte se ha trabajado con un único dron, el dominio se ha tipado de forma general para permitir su extensión posterior.

1.1.3. Predicados

Se han utilizado los siguientes predicados:

- at-drone(?d, ?l): el dron ?d se encuentra en la localización ?l.
- at-person(?p, ?l): la persona ?p se encuentra en ?l.
- at-crate(?c, ?l): la caja ?c se encuentra en la localización ?l.
- hand-empty(?d, ?a): el brazo ?a del dron ?d está libre.
- holding(?d, ?a, ?c): el dron ?d está sosteniendo la caja ?c con el brazo ?a.
- crate-content(?c, ?cnt): la caja ?c contiene el recurso ?cnt.
- has-content(?p, ?cnt): la persona ?p ya dispone del contenido ?cnt.

Este conjunto de predicados permite representar de manera suficiente el estado del mundo para resolver el problema. En particular, la combinación de hand-empty y holding refleja la disponibilidad de cada brazo, mientras que crate-content y has-content separan claramente la estructura logística del problema del contenido que interesa entregar.

1.1.4. Acciones del dominio

El dominio contiene tres acciones: fly, pick-up y deliver.

Acción fly

La acción fly mueve un dron desde una localización origen a una localización destino. Su precondición exige que el dron se encuentre en la localización de salida y su efecto actualiza su posición.

```
(:action fly
  :parameters (?d - drone ?from - location ?to - location)
  :precondition (at-drone ?d ?from)
  :effect (and (not (at-drone ?d ?from)) (at-drone ?d ?to))
)
```

Esta acción modela el desplazamiento básico del dron entre dos puntos del mapa. En esta primera parte se ha asumido que el dron puede volar directamente entre cualquier par de localizaciones.

Acción pick-up

La acción pick-up permite recoger una caja situada en la misma localización que el dron, siempre que el brazo correspondiente esté libre.

```
(:action pick-up
  :parameters (?d - drone ?a - arm ?c - crate ?l - location)
  :precondition (and (at-drone ?d ?l) (at-crate ?c ?l) (hand-empty ?d ?a))
  :effect (and (not (at-crate ?c ?l)) (not (hand-empty ?d ?a)) (holding ?d ?a ?c))
)
```

Esta acción garantiza que una caja no pueda estar simultáneamente en el suelo y en la mano del dron. Además, hace explícito el uso de cada brazo, lo que permite transportar hasta dos cajas de forma simultánea.

Acción deliver

La acción deliver representa la entrega del contenido de una caja a una persona situada en la misma localización que el dron. Para que pueda aplicarse, el dron debe estar sosteniendo la caja, la persona debe estar presente en esa localización y la caja debe contener el recurso requerido.

```
(:action deliver
  :parameters (?d - drone ?a - arm ?c - crate ?p - person ?l - location ?cnt - content)
  :precondition (and (at-drone ?d ?l) (at-person ?p ?l) (holding ?d ?a ?c) (crate-content ?c ?cnt))
  :effect (and (not (holding ?d ?a ?c)) (hand-empty ?d ?a) (has-content ?p ?cnt))
)
```

Con esta acción se ha optado por una abstracción típica en planificación clásica: el efecto relevante no es el destino posterior de la caja, sino el hecho de que la persona ya posee el contenido. Dado que las metas del problema se formulan precisamente en términos de has-content, esta representación resulta suficiente para la Parte 1.

1.1.5. Justificación del modelado

El dominio propuesto cumple correctamente con el espíritu del enunciado y con las restricciones del formalismo STRIPS. Se trata de un modelo sencillo, pero expresivo, en el que:

- se distingue entre localización del dron, localización de las cajas y localización de las personas;
- se representa explícitamente la capacidad del dron para usar dos brazos;
- se modela el contenido de forma genérica;
- y las metas se formulan sobre el resultado de la entrega, no sobre el movimiento en sí.

Además, el dominio es lo bastante compacto como para que los planificadores puedan resolver instancias de tamaño creciente, lo cual es especialmente importante en las fases experimentales de la práctica.

1.1.6. Problemas de prueba

Para validar el dominio se construyeron dos problemas pequeños a mano.

Problema 1

En el primer problema se considera un único dron, una caja, una persona y dos localizaciones: el depósito y una localización de destino. La caja contiene comida y la meta consiste en que la persona reciba ese contenido. Este problema sirve para comprobar el funcionamiento básico de las tres acciones del dominio: recoger una caja, volar hasta el destino y realizar la entrega.

Problema 2

En el segundo problema se introducen dos personas, tres cajas y dos contenidos distintos (food y medicine). El dron debe entregar comida a una persona y medicina a otra. Este segundo caso es más representativo, ya que obliga a decidir qué cajas utilizar y permite comprobar que el dominio soporta objetivos múltiples.

La elección de tres cajas para satisfacer dos objetivos es deliberada, ya que refleja que no siempre será necesario transportar todas las cajas disponibles en el depósito. Esta posibilidad está alineada con el enunciado y hace que el dominio sea más realista.

1.2. Generación automática de problemas y benchmark con FF

1.2.1. Generador de problemas

Una vez validado el dominio con los dos ejemplos pequeños, se utilizó un generador automático de problemas en Python para crear instancias de mayor tamaño. El objetivo del generador es producir problemas estructuralmente homogéneos, pero con suficiente variación como para analizar el comportamiento de los planificadores conforme crece el tamaño de la entrada.

El generador crea automáticamente:

- un depósito y un conjunto de localizaciones;
- un dron;
- un conjunto de personas situadas en distintas localizaciones;
- un conjunto de cajas inicialmente colocadas en el depósito;
- contenidos asociados a las cajas;
- y una meta formada por varias necesidades de las personas.

Para los experimentos de esta parte se ha trabajado con instancias de la forma problem_d1_r0_lN_pN_cN_gN.pddl, manteniendo siempre:

- 1 dron,
- 0 transportadores,
- N localizaciones,
- N personas,
- N cajas,
- y N metas.

De esta forma, el parámetro N se utiliza como medida global del tamaño del problema.

1.2.2. Metodología experimental con FF

El primer planificador empleado fue FF, con el objetivo de determinar hasta qué tamaño de problema podía resolver el dominio en un tiempo máximo de 60 segundos.

La estrategia seguida fue en dos fases:

1. Barrido grueso de 5 en 5, para localizar aproximadamente el umbral de resolución.
2. Refinamiento final de 1 en 1 en el último tramo antes del primer timeout, para determinar con mayor precisión el tamaño máximo resoluble.

Este procedimiento permite equilibrar coste experimental y precisión. Un barrido exclusivamente unitario desde tamaños pequeños habría sido innecesariamente lento, mientras que un barrido solo en saltos de cinco habría dado una frontera demasiado grosera.

1.2.3. Resultados del benchmark con FF

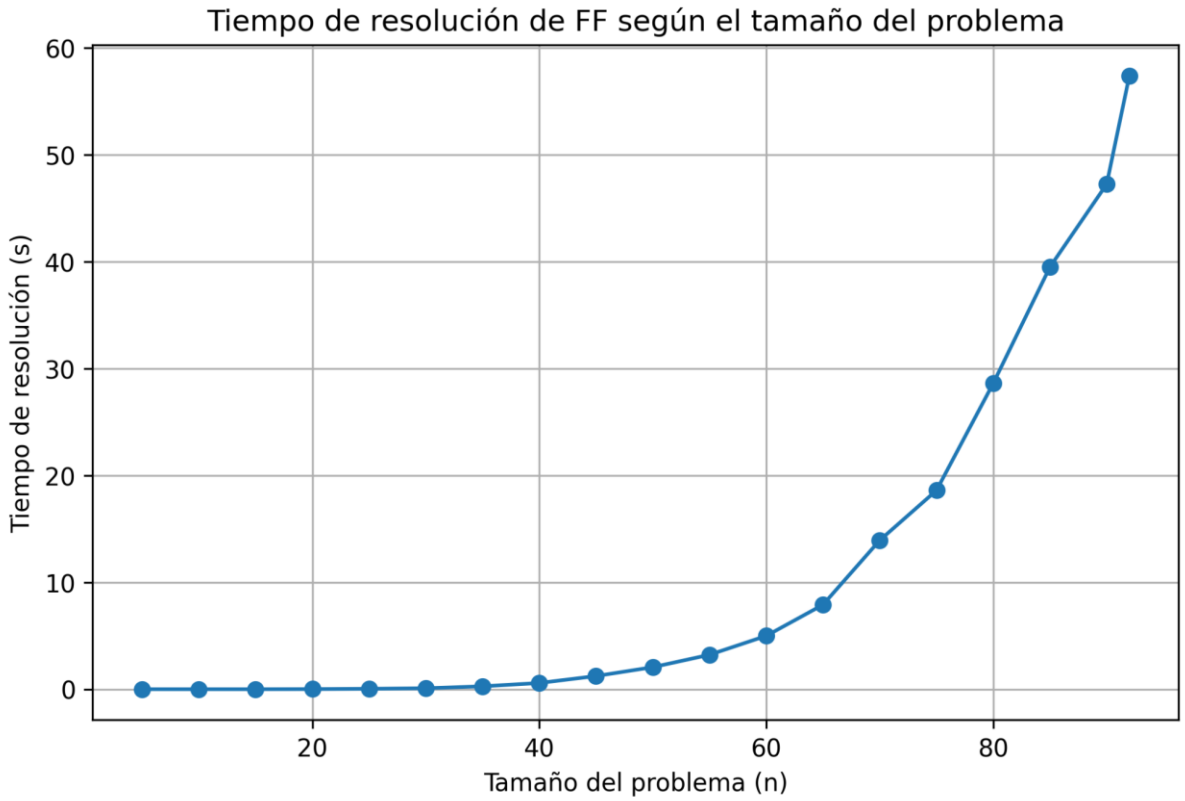
Los resultados muestran que FF resuelve con gran rapidez los tamaños pequeños y medianos, pero su tiempo crece de forma acusada al aumentar N. Tras el refinamiento final, el mayor tamaño resuelto dentro del límite de 60 segundos fue:

N = 92

La evolución observada es la esperable: para valores bajos de N, el tiempo de resolución es prácticamente despreciable, pero a partir de cierto punto el crecimiento se acelera de forma muy marcada.

La siguiente figura recoge la evolución del tiempo de resolución de FF en función del tamaño del problema.

Figura 1. Tiempo de resolución de FF según el tamaño del problema.



A partir de los datos experimentales, se aprecia una tendencia claramente creciente y no lineal. La zona crítica aparece aproximadamente entre N = 75 y N = 92, donde incrementos pequeños en el tamaño del problema producen aumentos significativos en el tiempo necesario para encontrar un plan.

Conviene señalar que la dificultad no crece de manera estrictamente monótona con N, ya que las instancias generadas para cada tamaño no son idénticas entre sí. Por ello, pueden aparecer pequeñas oscilaciones en los tiempos observados. Aun así, la tendencia global es muy clara y permite extraer una conclusión robusta: FF se comporta muy bien en este dominio para tamaños moderados, pero termina alcanzando un umbral a partir del cual el crecimiento del espacio de búsqueda se vuelve dominante.

1.2.4. Discusión de los resultados de FF

El buen rendimiento de FF en este dominio se explica por la propia naturaleza del planificador. FF utiliza una heurística basada en un problema relajado y combina Enforced Hill-Climbing con una búsqueda adicional cuando es necesario. En dominios relativamente estructurados y con acciones bastante directas, como el propuesto en esta primera parte, este enfoque suele proporcionar una resolución muy rápida.

Sin embargo, conforme el número de personas, cajas y metas aumenta, también crece el número de posibles combinaciones entre recogidas, vuelos y entregas. El dominio sigue siendo compacto, pero el espacio de estados se hace mucho más grande, y esa ampliación del espacio termina afectando al tiempo de búsqueda.

Por tanto, los resultados de esta sección confirman dos ideas importantes. La primera es que el dominio está bien modelado y es perfectamente resoluble por un planificador clásico. La segunda es que, incluso con un dominio relativamente simple, el crecimiento del tamaño del problema impacta de forma notable en la complejidad práctica de la planificación.

1.3. Comparativa con Pyperplan

1.3.1. Comparación entre BFS, IDS, A* y GBFS

En este apartado se compararon varios algoritmos disponibles en Pyperplan con un límite máximo de 60 segundos por instancia. Los algoritmos estudiados fueron:

- BFS
- IDS
- A* con hMAX
- GBFS con hMAX

La metodología utilizada fue similar a la del apartado anterior, aunque con una diferencia importante: para evitar descartar un algoritmo por una única instancia especialmente desfavorable, se permitió un margen de dos fallos consecutivos antes de dejar de evaluarlo. Además, cuando un problema ya había sido generado previamente, se reutilizó el archivo correspondiente para mantener la comparabilidad entre ejecuciones.

Los resultados finales fueron los siguientes:

Algoritmo	Mayor tamaño resuelto (N)	Problema de referencia	Tiempo en esa instancia (s)	Nº acciones del plan	¿Plan óptimo?
BFS	7	problem_d1_r0_l7_p7_c7_g7.pddl	1.20	21	Sí
IDS	4	problem_d1_r0_l4_p4_c4_g4.pddl	3.60	12	Sí
A* + hMAX	6	problem_d1_r0_l6_p6_c6_g6.pddl	3.40	18	Sí
GBFS + hMAX	8	problem_d1_r0_l8_p8_c8_g8.pddl	1.90	29	No garantizado

Estos resultados reflejan diferencias claras entre estrategias de búsqueda.

BFS obtiene un comportamiento razonable en tamaños pequeños y medianos. Dado que todas las acciones tienen coste uniforme en esta Parte 1, BFS garantiza encontrar planes de longitud mínima, pero su consumo de memoria crece rápidamente y eso limita su escalabilidad.

IDS presenta el peor rendimiento de los cuatro algoritmos analizados. Aunque conserva la ventaja teórica de utilizar menos memoria que BFS, el coste de repetir búsquedas a profundidad creciente termina penalizando de forma notable su tiempo de ejecución en este dominio.

A* con hMAX alcanza un tamaño máximo intermedio. La heurística hMAX es admisible, por lo que A* mantiene optimalidad, pero esa garantía se paga con un mayor esfuerzo de búsqueda que en el caso de estrategias puramente voraces.

GBFS con hMAX es el algoritmo que llega más lejos, alcanzando tamaño 8. Esto es coherente con la naturaleza de GBFS: prioriza rapidez y orientación heurística, aunque renuncia a garantías de optimalidad.

En conjunto, esta comparativa muestra el equilibrio clásico entre optimalidad, memoria y velocidad. Los algoritmos informados y voraces pueden resolver instancias algo mayores, mientras que los métodos óptimos o sistemáticos puros ven limitada antes su cobertura.

1.3.2. Comparación entre GBFS y EHC con distintas heurísticas

Una vez identificado en el apartado anterior que GBFS con hMAX era el mejor algoritmo satisficing de los considerados, se tomó como problema de referencia el mayor tamaño que este pudo resolver en menos de 60 segundos, es decir:

problem_d1_r0_l8_p8_c8_g8.pddl

Sobre esta instancia se compararon:

- GBFS
- EHC

con las heurísticas:

- hMAX
- hADD
- hFF
- Landmark

Los resultados obtenidos fueron los siguientes:

Búsqueda	Heurística	Resuelto	Tiempo (s)	Longitud del plan
GBFS	hMAX	Sí	2.1	29
GBFS	hADD	Sí	0.089	30
GBFS	hFF	Sí	0.15	29
GBFS	Landmark	Sí	0.0047	31
EHC	hMAX	No (timeout)	—	—
EHC	hADD	Sí	0.38	34
EHC	hFF	Sí	0.48	32
EHC	Landmark	Sí	0.014	36

La primera conclusión es que GBFS domina claramente a EHC en esta instancia. Resuelve todas las combinaciones heurísticas, mientras que EHC no logra resolver el caso con hMAX dentro del límite temporal. Además, cuando ambos resuelven, GBFS obtiene en general mejores tiempos y planes de menor longitud.

Respecto a las heurísticas, pueden extraerse varias observaciones:

- Landmark ofrece los mejores tiempos tanto con GBFS como con EHC, pero produce planes más largos.
- hFF proporciona un compromiso especialmente bueno en GBFS, ya que mantiene una longitud de plan tan buena como hMAX y un tiempo muy inferior.
- hADD también resulta efectiva, aunque en esta instancia queda algo por detrás de hFF.
- hMAX es la peor heurística para EHC y, aunque en GBFS obtiene buen plan, su coste temporal es considerablemente mayor.

Desde el punto de vista teórico, estos resultados son razonables. hMAX es admisible, pero suele ser menos informativa que heurísticas más agresivas como hADD o hFF. En cambio, hFF y Landmark no garantizan optimalidad, pero orientan mejor la búsqueda en términos prácticos. Eso explica que puedan dar tiempos mucho menores aun a costa de obtener planes algo peores.

En definitiva, para este dominio y este tamaño, la combinación más equilibrada entre rapidez y calidad del plan es GBFS con hFF, mientras que GBFS con Landmark destaca por pura velocidad.

1.3.3. Comparación entre BFS, IDS y A* con heurísticas admisibles

Para el estudio de algoritmos óptimos se tomó como referencia el mayor tamaño resuelto por A* con hMAX en el apartado 1.3.1, es decir:

problem_d1_r0_l6_p6_c6_g6.pddl

Sobre esta instancia se compararon:

- BFS
- IDS
- A* con hMAX
- A* con LM-Cut

Los resultados fueron los siguientes:

Algoritmo	Resuelto	Tiempo (s)	Longitud del plan
BFS	Sí	1.7	18
IDS	No (timeout)	—	—
A* + hMAX	Sí	3.4	18
A* + LM-Cut	No (timeout)	—	—

La principal observación es que BFS y A* con hMAX encuentran planes de la misma longitud, concretamente 18, lo que es coherente con el hecho de que ambos son métodos óptimos en este contexto. Sin embargo, BFS lo hace en menos tiempo que A* con hMAX para esta instancia concreta.

IDS vuelve a mostrar el peor comportamiento y no consigue resolver el problema dentro del tiempo límite. Este resultado confirma la tendencia ya observada en 1.3.1: aunque IDS es atractivo desde el punto de vista teórico por su bajo consumo de memoria, en la práctica su repetición de búsquedas lo penaliza mucho.

El caso de A* con LM-Cut es especialmente interesante. LM-Cut es una heurística admisible y teóricamente muy potente, pero en esta instancia no logra completar la búsqueda dentro del tiempo máximo. Esto no significa que la heurística sea mala, sino que su mayor coste de cálculo puede llegar a ser contraproducente en problemas pequeños o medianos donde el overhead de la propia heurística pesa demasiado.

La conclusión de este apartado es que, en el dominio de esta Parte 1, la superioridad teórica de una heurística más informada no siempre se traduce en un mejor tiempo de resolución real. En ciertas instancias, una búsqueda sistemática simple como BFS puede competir e incluso superar a enfoques heurísticos óptimos más sofisticados.

1.4. Conclusiones de la Parte 1

La primera parte de la práctica ha permitido construir, validar y evaluar un dominio clásico de planificación para reparto mediante drones. El trabajo realizado permite extraer varias conclusiones relevantes.

En primer lugar, el dominio emergency-services modela correctamente el problema planteado en el enunciado dentro de las restricciones de PDDL clásico con STRIPS y typing. La representación mediante contenidos genéricos y dos brazos resulta adecuada, flexible y suficientemente expresiva para la tarea propuesta.

En segundo lugar, el generador automático ha permitido escalar desde ejemplos pequeños construidos a mano hasta instancias mucho mayores, lo que ha hecho posible estudiar el comportamiento de distintos planificadores de forma experimental.

En tercer lugar, los resultados con FF muestran que el dominio es resoluble a tamaños bastante grandes, alcanzando hasta N = 92 en menos de un minuto. Esto indica que, aunque el espacio de búsqueda crece significativamente, la estructura del problema sigue siendo favorable para heurísticas eficaces.

Finalmente, la comparación con Pyperplan confirma varios comportamientos esperables desde la teoría:

- GBFS prioriza rapidez y cobertura, llegando más lejos que los métodos óptimos.
- IDS se comporta peor en la práctica debido al coste acumulado de iterar profundidades.
- A* ofrece un punto intermedio entre información heurística y garantías de optimalidad.
- La elección de la heurística es determinante: una heurística más admisible no siempre implica mejor rendimiento práctico, y heurísticas no admisibles como hFF o Landmark pueden resultar mucho más competitivas en contextos satisficing.

En conjunto, la Parte 1 no solo valida el modelado del dominio, sino que también muestra de forma clara la relación entre representación, heurísticas y comportamiento experimental de los planificadores.

1.5. Resumen final de resultados

Para facilitar la lectura, se resumen aquí los resultados más importantes de la Parte 1.

FF

Planificador	Mayor tamaño resuelto en 60 s
FF	92

Pyperplan: comparación inicial

Algoritmo	Mayor tamaño resuelto en 60 s
BFS	7
IDS	4
A* + hMAX	6
GBFS + hMAX	8

GBFS y EHC con distintas heurísticas sobre l8

Búsqueda	Heurística	Tiempo (s)	Longitud
GBFS	hMAX	2.1	29
GBFS	hADD	0.089	30
GBFS	hFF	0.15	29
GBFS	Landmark	0.0047	31
EHC	hMAX	timeout	—
EHC	hADD	0.38	34
EHC	hFF	0.48	32
EHC	Landmark	0.014	36

Algoritmos óptimos sobre l6

Algoritmo	Tiempo (s)	Longitud
BFS	1.7	18
IDS	timeout	—
A* + hMAX	3.4	18
A* + LM-Cut	timeout	—

PARTE 2.1. AMPLIACIÓN DEL DOMINIO CON TRANSPORTADORES

2.1.1. Objetivo de la ampliación

En esta segunda parte se amplía el dominio desarrollado en la Parte 1 para incorporar transportadores como mecanismo adicional de transporte. La idea fundamental del cambio es que el dron deje de disponer de dos brazos independientes y pase a tener una única mano con la que puede transportar directamente una sola caja. Para compensar esta reducción de capacidad directa, se introduce la posibilidad de mover un transportador y almacenar varias cajas en él.

El objetivo de esta ampliación es doble. Por un lado, enriquecer el dominio incorporando una estructura logística más realista que el simple transporte directo con dos brazos. Por otro, estudiar cómo afecta este cambio de modelado al comportamiento de los planificadores utilizados en la Parte 1.

El nuevo dominio mantiene la misma filosofía general del problema original: un dron debe repartir contenidos desde un depósito hasta distintas personas distribuidas por varias localizaciones. Sin embargo, ahora se añade una capa adicional de complejidad, ya que el dron puede:

- coger una caja con su única mano,
- introducirla en un transportador,
- desplazar el transportador a otra localización,
- sacar una caja del transportador,
- y entregar su contenido a una persona.

De esta manera, la ampliación no consiste solo en añadir nuevas acciones, sino en transformar de forma significativa la estructura del espacio de búsqueda.

2.1.2. Cambios respecto al dominio de la Parte 1

La diferencia más importante respecto al dominio anterior es la sustitución del modelo basado en dos brazos por otro basado en:

- una única mano, y
- uno o varios transportadores con capacidad limitada.

En la Parte 1 el dron podía sostener hasta dos cajas simultáneamente, una en cada brazo. En 2.1 esa posibilidad desaparece. El dron pasa a tener un único estado de mano libre u ocupada, representado mediante los predicados hand-empty y holding.

Para no perder capacidad de transporte, se introduce un nuevo tipo de objeto, transporter, que puede contener varias cajas. La capacidad del transportador no se modela mediante posiciones individuales o “ranuras” concretas, sino mediante una representación agregada del número de huecos libres. Esta decisión de modelado evita estados redundantes y hace el dominio más limpio.

Además, se introduce el tipo num, que se utiliza para representar los distintos valores posibles de capacidad libre, junto con el predicado siguiente, que permite codificar la relación entre números consecutivos.

2.1.3. Tipos y predicados del nuevo dominio

El dominio ampliado define los siguientes tipos:

- location
- person
- drone
- crate
- content
- transporter
- num

En comparación con la Parte 1, desaparece el tipo arm y se añaden transporter y num.

Los predicados del dominio son los siguientes:

- at-drone(?d, ?l): el dron ?d está en la localización ?l.
- at-person(?p, ?l): la persona ?p está en la localización ?l.
- at-crate(?c, ?l): la caja ?c está en la localización ?l.
- at-transporter(?t, ?l): el transportador ?t está en la localización ?l.
- hand-empty(?d): el dron ?d tiene la mano libre.
- holding(?d, ?c): el dron ?d está sosteniendo la caja ?c.
- crate-content(?c, ?cnt): la caja ?c contiene ?cnt.
- has-content(?p, ?cnt): la persona ?p ya dispone del contenido ?cnt.
- in-transporter(?c, ?t): la caja ?c está dentro del transportador ?t.
- free-slots(?t, ?n): el transportador ?t dispone actualmente de ?n huecos libres.
- siguiente(?n1, ?n2): el valor ?n2 es el sucesor de ?n1.

Este modelado permite representar la logística del transportador sin necesidad de individualizar sus posiciones internas. En lugar de modelar “la caja 1 va en la ranura A”, “la caja 2 va en la ranura B”, etc., se trabaja directamente con el número de huecos disponibles, que es justo la información relevante para controlar la capacidad.

2.1.4. Acciones del dominio

El dominio ampliado incorpora cinco acciones principales, además del movimiento básico del dron.

Acción fly

La acción fly mantiene el desplazamiento directo del dron entre localizaciones.

```
(:action fly
  :parameters (?d - drone ?from - location ?to - location)
  :precondition (at-drone ?d ?from)
  :effect (and
    (not (at-drone ?d ?from))
    (at-drone ?d ?to)
  )
)
```

A diferencia de algunas variantes posibles, esta acción no exige que la mano del dron esté vacía. Por tanto, el dron puede volar mientras sostiene una caja, lo cual preserva la posibilidad de transporte directo de una única caja.

Acción pick-up

Esta acción permite al dron recoger una caja si está en la misma localización y tiene la mano libre.

```
(:action pick-up
  :parameters (?d - drone ?c - crate ?l - location)
  :precondition (and
    (at-drone ?d ?l)
    (at-crate ?c ?l)
    (hand-empty ?d)
  )
  :effect (and
    (not (at-crate ?c ?l))
    (not (hand-empty ?d))
    (holding ?d ?c)
  )
)
```

Con ello se mantiene la coherencia del estado: una caja no puede estar simultáneamente en el suelo y en la mano del dron.

Acción deliver

La acción deliver sigue modelando la entrega del contenido de una caja a una persona situada en la misma localización.

```
(:action deliver
  :parameters (?d - drone ?c - crate ?p - person ?l - location ?cnt - content)
  :precondition (and
    (at-drone ?d ?l)
    (at-person ?p ?l)
```

```
    (holding ?d ?c)
    (crate-content ?c ?cnt)
  )
  :effect (and
    (not (holding ?d ?c))
    (hand-empty ?d)
    (has-content ?p ?cnt)
  )
)
```

Igual que en la Parte 1, se ha optado por una abstracción donde el efecto relevante es que la persona obtiene el contenido, ya que ese es el hecho que aparece en las metas.

Acción put-in-transporter

Esta acción permite introducir una caja en un transportador situado en la misma localización, siempre que el dron esté sujetando la caja y el transportador tenga huecos libres.

```
(:action put-in-transporter
  :parameters (?d - drone ?c - crate ?t - transporter ?l - location ?n-new - num ?n-old - num)
  :precondition (and
    (at-drone ?d ?l)
    (at-transporter ?t ?l)
    (holding ?d ?c)
    (free-slots ?t ?n-old)
    (siguiente ?n-new ?n-old)
  )
  :effect (and
    (not (holding ?d ?c))
    (hand-empty ?d)
    (in-transporter ?c ?t)
    (not (free-slots ?t ?n-old))
    (free-slots ?t ?n-new)
  )
)
```

Aquí se observa claramente el papel de free-slots y siguiente: al introducir una caja, el número de huecos libres disminuye en una unidad.

Acción move-transporter

Esta acción modela el desplazamiento conjunto del dron y el transportador.

```
(:action move-transporter
  :parameters (?d - drone ?t - transporter ?from - location ?to - location)
  :precondition (and
    (at-drone ?d ?from)
    (at-transporter ?t ?from)
    (hand-empty ?d)
  )
  :effect (and
    (not (at-drone ?d ?from))
    (at-drone ?d ?to)
```

```
    (not (at-transporter ?t ?from))
    (at-transporter ?t ?to)
  )
)
```

Se exige que el dron tenga la mano libre, lo cual impide mover el transportador mientras se transporta una caja directamente. Esta restricción resulta coherente con la interpretación física del dominio.

Acción take-out-transporter

La acción take-out-transporter permite extraer una caja del transportador cuando el dron tiene la mano libre.

```
(:action take-out-transporter
  :parameters (?d - drone ?c - crate ?t - transporter ?l - location ?n-old - num ?n-new - num)
  :precondition (and
    (at-drone ?d ?l)
    (at-transporter ?t ?l)
    (in-transporter ?c ?t)
    (hand-empty ?d)
    (free-slots ?t ?n-old)
    (siguiente ?n-old ?n-new)
  )
  :effect (and
    (not (in-transporter ?c ?t))
    (not (hand-empty ?d))
    (holding ?d ?c)
    (not (free-slots ?t ?n-old))
    (free-slots ?t ?n-new)
  )
)
```

En este caso el número de huecos libres aumenta en una unidad, ya que una caja abandona el transportador.

2.1.5. Justificación del modelado

El aspecto más importante de este dominio no es solo la presencia del transportador, sino **cómo** se representa su capacidad. En lugar de modelar posiciones internas individualizadas, se ha elegido una representación agregada mediante el número de huecos libres. Esta decisión simplifica el dominio y evita introducir diferencias artificiales entre estados equivalentes.

Por ejemplo, si un transportador tiene capacidad 4 y contiene dos cajas, desde el punto de vista del problema lo relevante no es en qué “posición interna” está cada caja, sino que aún quedan 2 huecos libres. Modelar las posiciones concretas habría incrementado el número de estados sin aportar información útil para la resolución.

Además, el uso de una sola mano en lugar de dos brazos fuerza una dependencia real del transportador. En la Parte 1 el dron podía resolver muchos problemas transportando dos cajas directamente. En esta Parte 2.1 la única manera de conservar una cierta capacidad de agrupación es utilizar el transportador de forma explícita.

En consecuencia, el nuevo dominio introduce más estructura logística, más decisiones intermedias y, en general, un espacio de búsqueda más complejo.

2.1.6. Adaptación de los problemas experimentales

El enunciado de esta parte pedía repetir los experimentos equivalentes a los apartados 1.3.2 y 1.3.3 de la Parte 1, pero adaptando los problemas utilizados al nuevo dominio.

Por ello, no se generaron instancias aleatorias nuevas para esos dos apartados, sino que se adaptaron los problemas de referencia usados previamente:

- para la repetición del apartado equivalente a 1.3.2, se utilizó la versión adaptada de problem_d1_r0_l8_p8_c8_g8
- para la repetición del apartado equivalente a 1.3.3, se utilizó la versión adaptada de problem_d1_r0_l6_p6_c6_g6

La adaptación consistió en:

- cambiar el dominio de referencia al nuevo emergency-services-transporters,
- eliminar los objetos de tipo arm,
- sustituir la representación de dos brazos por una sola mano,
- añadir un transportador,
- añadir los objetos numéricos necesarios para representar la capacidad,
- e inicializar la relación siguiente y el número de huecos libres.

Con ello se garantiza que la comparación entre la Parte 1 y la Parte 2.1 se realiza sobre el mismo caso base, pero bajo un dominio diferente.

2.1.7. Repetición del experimento equivalente a 1.3.2

En la Parte 1 se había comparado el comportamiento de GBFS y EHC con distintas heurísticas sobre el problema fijo de tamaño 8. En esta parte se ha repetido ese estudio sobre el problema adaptado al nuevo dominio con transportadores.

La instancia empleada fue:

problem_d1_r1_l8_p8_c8_g8.pddl

Las combinaciones comparadas fueron:

- GBFS + hMAX
- GBFS + hADD
- GBFS + hFF
- GBFS + Landmark
- EHC + hMAX
- EHC + hADD
- EHC + hFF
- EHC + Landmark

Los resultados obtenidos fueron los siguientes:

Búsqueda	Heurística	Resuelto	Tiempo (s)	Longitud del plan
GBFS	hMAX	No (timeout)	—	—
GBFS	hADD	Sí	3.6	36
GBFS	hFF	Sí	0.2	31
GBFS	Landmark	No (timeout)	—	—
EHC	hMAX	No (timeout)	—	—
EHC	hADD	Sí	0.54	32
EHC	hFF	Sí	1.0	36
EHC	Landmark	Sí	0.046	36

La primera conclusión es inmediata: el nuevo dominio es significativamente más difícil que el de la Parte 1. Combinaciones que antes resolvían cómodamente la instancia ahora no lo consiguen dentro del límite temporal. Esto se observa especialmente en hMAX, que hace timeout tanto con GBFS como con EHC.

También se aprecia que la interacción entre heurística y estrategia de búsqueda se vuelve más compleja. En la Parte 1, por ejemplo, GBFS con Landmark resolvía muy rápido; en esta nueva versión, esa combinación no logra completar la búsqueda dentro del tiempo límite. En cambio, EHC + Landmark sí resuelve y además lo hace con un tiempo muy bajo. Este comportamiento, aunque llamativo, es razonable: una misma heurística puede comportarse de forma muy distinta según el algoritmo de búsqueda que la explote. En presencia de mesetas heurísticas o grandes conjuntos de estados con valores similares, una búsqueda voraz best-first puede atascarse mientras un procedimiento de ascenso local logra progresar.

En términos globales, las mejores combinaciones en esta instancia son:

- GBFS + hFF, por su muy buen equilibrio entre tiempo y longitud de plan;
- EHC + hADD, por producir un plan algo más corto que EHC + hFF;
- EHC + Landmark, por su rapidez extrema, aunque con un plan más largo.

La comparación con la Parte 1 muestra con claridad que el añadido del transportador hace el dominio más exigente y más sensible a la combinación concreta de estrategia de búsqueda y heurística.

2.1.8. Repetición del experimento equivalente a 1.3.3

También se repitió el experimento análogo al apartado 1.3.3 de la Parte 1, esta vez utilizando el problema adaptado de tamaño 6.

La instancia empleada fue:

problem_d1_r1_l6_p6_c6_g6.pddl

Se compararon los algoritmos:

- BFS
- IDS
- A* con hMAX
- A* con LM-Cut

Los resultados fueron:

Algoritmo	**Resuelto**	**Tiempo (s)**	**Longitud del plan**
BFS	Sí	3.9	23
IDS	No (timeout)	—	—
A* + hMAX	No (timeout)	—	—
A* + LM-Cut	No (timeout)	—	—

Aquí la conclusión es aún más clara: la ampliación del dominio reduce drásticamente la eficacia de los algoritmos óptimos considerados. Mientras que en la Parte 1 tanto BFS como A* con hMAX resolvían la instancia de referencia, en esta nueva versión solo BFS consigue hacerlo dentro del límite temporal.

Esto puede explicarse por varios factores. Por un lado, el dominio ahora contiene más acciones y más decisiones intermedias. Ya no basta con elegir qué caja recoger y dónde entregarla; también hay que decidir cuándo introducir cajas en el transportador, cuándo moverlo y cuándo extraerlas. Por otro lado, las heurísticas admisibles como hMAX o LM-Cut tienen un coste de cálculo adicional que, en este dominio ampliado, parece penalizar más de lo que ayuda.

El resultado de LM-Cut es especialmente ilustrativo. En teoría es una heurística muy potente, pero en esta instancia no llega a resolver dentro del tiempo máximo. Esto refuerza una idea importante: una heurística teóricamente más informada no garantiza automáticamente un mejor comportamiento práctico cuando el coste de su cómputo es alto.

La comparación con la Parte 1 confirma, por tanto, que el nuevo dominio no solo es más complejo, sino que además dificulta especialmente a los algoritmos óptimos.

2.1.9. Discusión global de resultados

La principal conclusión experimental de la Parte 2.1 es que la introducción de transportadores aumenta de forma apreciable la complejidad práctica del problema, incluso cuando el tamaño de las instancias de referencia se mantiene igual que en la Parte 1.

Este aumento de complejidad se debe a varios motivos:

- el dron solo dispone de una mano, por lo que pierde capacidad directa de carga;
- aparecen nuevas acciones intermedias (put-in-transporter, move-transporter, take-out-transporter);
- la secuencia de decisiones se hace más larga;
- y el espacio de búsqueda crece por la necesidad de coordinar el uso del transportador con los movimientos del dron.

Los resultados también muestran que la heurística adecuada depende más que nunca de la estrategia de búsqueda. En la Parte 1 las diferencias entre combinaciones ya eran apreciables, pero en esta Parte 2.1 aparecen comportamientos mucho más contrastados, incluyendo combinaciones que pasan de ser muy eficaces a no resolver la instancia dentro del tiempo límite.

Desde el punto de vista metodológico, esta parte confirma que una ampliación aparentemente razonable del dominio puede cambiar de forma muy significativa el rendimiento de los planificadores, incluso cuando el objetivo final del problema sigue siendo esencialmente el mismo.

2.1.10. Conclusiones de la Parte 2.1

La ampliación del dominio con transportadores ha permitido enriquecer de forma sustancial el problema de reparto modelado en la Parte 1. El nuevo dominio representa una situación más compleja y más realista, en la que el dron debe combinar transporte directo y transporte agrupado mediante un contenedor con capacidad limitada.

El modelado elegido, basado en una sola mano y en una representación agregada de la capacidad del transportador, resulta adecuado y evita sobrecargar innecesariamente el dominio con detalles irrelevantes. Además, la adaptación de los problemas de referencia de la Parte 1 permite hacer comparaciones experimentales directas entre ambos dominios.

Los resultados empíricos muestran con claridad que el nuevo dominio es más difícil de resolver. Las combinaciones satisficing siguen ofreciendo mejores resultados que las óptimas, pero incluso entre las primeras aparecen más timeouts y una mayor sensibilidad a la heurística empleada. Entre los algoritmos óptimos, la degradación es todavía más visible, hasta el punto de que solo BFS resuelve la instancia de referencia de tamaño 6.

En conjunto, la Parte 2.1 demuestra que la forma en que se modela un problema de planificación influye de manera decisiva en el rendimiento de los planificadores. No basta con que el dominio sea correcto desde el punto de vista lógico; también importa enormemente cómo crece el espacio de búsqueda que induce.

2.1.11. Resumen final de resultados

Repetición del experimento tipo 1.3.2 sobre I8

Búsqueda	Heurística	Tiempo (s)	Longitud	Resultado
GBFS	hMAX	—	—	timeout
GBFS	hADD	3.6	36	resuelto
GBFS	hFF	0.2	31	resuelto
GBFS	Landmark	—	—	timeout
EHC	hMAX	—	—	timeout
EHC	hADD	0.54	32	resuelto
EHC	hFF	1.0	36	resuelto
EHC	Landmark	0.046	36	resuelto

Repetición del experimento tipo 1.3.3 sobre I6

Algoritmo	Tiempo (s)	Longitud	Resultado
BFS	3.9	23	resuelto
IDS	—	—	timeout
A* + hMAX	—	—	timeout
A* + LM-Cut	—	—	timeout

PARTE 2.2. PLANIFICACIÓN CON COSTES Y OPTIMIZACIÓN

2.2.1. Objetivo de la ampliación

En esta última parte se amplía el dominio desarrollado en 2.1 para introducir costes de acción y permitir que los planificadores optimicen no solo la consecución de las metas, sino también el coste total del plan obtenido. El objetivo principal ya no es únicamente encontrar una secuencia válida de acciones, sino hacerlo minimizando el coste acumulado.

La motivación de esta ampliación es clara. En la versión anterior del dominio, todas las acciones se consideraban implícitamente equivalentes desde el punto de vista del planificador, por lo que el interés estaba centrado en la longitud del plan o en la capacidad de encontrar cualquier solución dentro del límite de tiempo. Sin embargo, en un problema logístico como este, no todas las acciones deberían tener el mismo peso. En particular, los desplazamientos entre localizaciones lejanas deben resultar más costosos que las operaciones locales de manipulación de cajas.

Por ello, en esta parte se ha incorporado una función de coste de vuelo y una métrica de minimización del coste total. De este modo, el dominio pasa de ser un problema clásico puramente secuencial a uno de planificación con optimización, donde la calidad del plan depende directamente del coste acumulado.

2.2.2. Cambios en el dominio

La principal modificación del dominio consiste en añadir el requisito:

```
(:requirements :strips :typing :action-costs)
```

Con ello se habilita el uso del formalismo restringido de costes de acción en PDDL.

Además, se introducen dos funciones:

```
(:functions
  (total-cost)
  (fly-cost ?from - location ?to - location)
)
```

La función total-cost representa el coste acumulado del plan, mientras que fly-cost(?from, ?to) representa el coste del desplazamiento entre dos localizaciones concretas.

A partir de aquí, el dominio modificado conserva la misma estructura general que en 2.1, pero todas las acciones pasan a contribuir al coste total.

2.2.3. Acciones y política de costes

Se ha adoptado una política de costes sencilla y coherente con la interpretación logística del problema.

Acciones con coste variable

Las acciones que implican desplazamiento físico entre localizaciones incrementan el coste total según la función fly-cost:

- fly
- move-transporter

Por ejemplo, la acción fly queda modelada así:

```
(:action fly
  :parameters (?d - drone ?from - location ?to - location)
  :precondition (at-drone ?d ?from)
  :effect (and
    (not (at-drone ?d ?from))
    (at-drone ?d ?to)
    (increase (total-cost) (fly-cost ?from ?to))
  )
)
```

Y de forma análoga, move-transporter añade al coste total el valor correspondiente al desplazamiento realizado.

Esta decisión refleja de forma natural que moverse entre localizaciones lejanas debe ser más caro que realizar operaciones locales, y que trasladar el transportador también implica un coste asociado al trayecto.

Acciones con coste constante

Las acciones locales de manipulación tienen un coste fijo igual a 1:

- pick-up
- deliver
- put-in-transporter
- take-out-transporter

Esto permite distinguir entre dos tipos de esfuerzo:

- coste elevado y dependiente de la distancia para los movimientos;
- coste pequeño y constante para las operaciones de carga, descarga y entrega.

El resultado es un modelo en el que el planificador puede verse incentivado a usar el transportador para reducir desplazamientos caros, aunque eso suponga introducir más acciones intermedias de coste bajo.

2.2.4. Generación de problemas con costes

El generador de problemas también se modificó para ser compatible con el nuevo dominio.

En particular, se añadieron tres cambios fundamentales:

1. Inicialización del coste total a cero:
   (= (total-cost) 0)

2. Inicialización de la función fly-cost para cada par de localizaciones, utilizando la función flight_cost() del generador. De esta forma, cada problema incluye explícitamente el coste de todos los posibles vuelos entre el depósito y las localizaciones, así como entre las distintas localizaciones entre sí.

3. Inclusión de la métrica:
   (:metric minimize (total-cost))

Con ello, cada instancia queda completamente preparada para ser resuelta por planificadores que soporten optimización.

Durante los experimentos se mantuvo siempre la misma configuración estructural básica utilizada en 2.1:

- 1 dron,
- 1 transportador,
- capacidad del transportador igual a 4.

Además, igual que en los apartados anteriores, se emplearon instancias de la forma problem_d1_r1_lN_pN_cN_gN.pddl, utilizando N como medida global del tamaño del problema.

2.2.5. Metodología experimental

El enunciado de esta parte pedía estudiar por separado dos familias de planificadores:

- planificadores satisficing, que buscan buenas soluciones rápidamente pero no garantizan optimalidad;
- planificadores óptimos, que deben certificar que la solución encontrada es de coste mínimo.

Para cada planificador se siguió la misma estrategia general:

1. generar una secuencia de problemas de tamaño creciente;
2. probar el planificador en cada tamaño con un límite de 60 segundos;
3. localizar aproximadamente la frontera de resolución con saltos de 5 en 5;
4. refinar el último tramo de 1 en 1 cuando aparecía el primer fallo;
5. registrar el mayor tamaño resuelto, el tiempo empleado, el coste del plan y su longitud.

Esta metodología permite obtener una comparación homogénea entre planificadores, manteniendo fijo el criterio de tiempo máximo y utilizando problemas generados con la misma estructura.

Una diferencia importante respecto a los apartados anteriores es que aquí la variable principal ya no es solo la longitud del plan, sino también su coste total, ya que el objetivo del problema es precisamente minimizar ese valor.

2.2.6. Planificadores satisficing evaluados

Se evaluaron los siguientes planificadores satisficing:

- Metric-FF
- Fast Downward con alias lama-first
- Fast Downward con alias seq-sat-fdss-2
- Fast Downward con alias seq-sat-fd-autotune-2

Los resultados obtenidos fueron los siguientes:

Planificador	Mayor tamaño resuelto	Coste de la solución	Longitud del plan	Tiempo total (s)
Metric-FF	10*	2081	41	0.00
lama-first	97	33638	414	38.20
seq-sat-fdss-2	6	1677	23	0.0036
seq-sat-fd-autotune-2	5	1769	23	17.47

* En el caso de Metric-FF, la ejecución dejó de ser comparable a partir de tamaño 11 debido a un error del propio planificador relacionado con la optimización ("unknown optimization method"). Por ello, el valor 10 debe interpretarse como el mayor tamaño resuelto correctamente antes de ese fallo, no como una frontera estricta obtenida por timeout.

Metric-FF
Metric-FF es una extensión de FF para planificación numérica con costes. Su versión orientada a calidad utiliza una búsqueda tipo weighted A* guiada por el coste estimado del plan relajado, y admite funciones objetivo lineales que puedan traducirse a costes de acción independientes del estado. Esto explica que pueda comportarse bien en problemas pequeños o medianos con estructura clara, aunque también puede verse limitado por cuestiones de soporte práctico de optimización según la versión utilizada.

lama-first
LAMA es una familia de planificadores basada en búsqueda heurística hacia delante que combina información de *landmarks* con una variante de la heurística FF. Su fortaleza está en usar subobjetivos obligatorios del problema junto con una heurística muy informativa, lo que suele darle una cobertura muy alta en planificación satisficing. Que en nuestros experimentos lama-first destaque claramente es coherente con ese diseño.

seq-sat-fdss-2
seq-sat-fdss-2 pertenece a la familia Fast Downward Stone Soup, que en la documentación oficial aparece como una configuración portfolio. Eso significa que no es un único método, sino una combinación de varias configuraciones ejecutadas con repartos de tiempo prefijados. Este detalle es importante porque la propia documentación advierte de que estas configuraciones están ligadas a presupuestos temporales tipo IPC y no deben interpretarse de forma idéntica cuando se usan con límites mucho menores, como 60 s.

seq-sat-fd-autotune-2
seq-sat-fd-autotune-2 es una configuración del IPC 2011 incluida en Fast Downward como alias estándar. Una forma prudente de explicarlo en la memoria es decir que se trata de una configuración ajustada para rendir bien en conjuntos amplios de benchmarks, no específicamente en este dominio. Por eso, en un único dominio y con presupuesto corto puede pagar más overhead que configuraciones más directas.

Discusión de resultados satisficing

La conclusión principal es que LAMA-first destaca de manera muy clara sobre el resto. Su cobertura experimental es muy superior, alcanzando tamaño 97, mientras que el resto de planificadores satisficing quedan muy por debajo. Este comportamiento resulta coherente con la naturaleza de LAMA-first, que combina heurísticas muy competitivas y estrategias de búsqueda orientadas a obtener soluciones de buena calidad en poco tiempo.

El rendimiento de Metric-FF es bueno en instancias pequeñas, pero no puede analizarse de forma completamente homogénea con el resto a partir de tamaño 11, ya que el planificador devuelve un error interno vinculado al tratamiento de la optimización. En consecuencia, aunque sus resultados iniciales son válidos, su cobertura global debe interpretarse con cautela.

Los dos aliases satisficing de Fast Downward restantes obtienen resultados mucho más modestos:

- seq-sat-fdss-2 alcanza tamaño 6,
- seq-sat-fd-autotune-2 se queda en tamaño 5.

Aunque a priori podría esperarse un rendimiento competitivo, en este dominio con costes el resultado muestra que esas configuraciones concretas no se adaptan tan bien como LAMA-first. En particular, el caso de seq-sat-fd-autotune-2 resulta llamativo, ya que incluso en tamaños muy pequeños presenta tiempos totales relativamente altos para la cobertura obtenida. Esto sugiere que el overhead de configuración o el comportamiento tipo portfolio pesa demasiado frente a la estructura concreta de este problema.

2.2.7. Planificadores óptimos evaluados

Para la búsqueda óptima se compararon los siguientes planificadores:

- Fast Downward con alias seq-opt-lmcut
- Fast Downward con alias seq-opt-bjolp
- Fast Downward con alias seq-opt-fdss-2

Los resultados fueron:

Planificador	Mayor tamaño resuelto	Coste óptimo	Longitud del plan	Tiempo total (s)
seq-opt-lmcut	6	610	29	29.32
seq-opt-bjolp	7	524	33	7.40
seq-opt-fdss-2	7	524	33	3.79

seq-opt-lmcut
seq-opt-lmcut es una configuración óptima basada en la heurística LM-Cut. Su atractivo es que ofrece una estimación admisible fuerte y, por tanto, mantiene optimalidad; su peaje es que el cálculo heurístico puede ser costoso. Esto encaja con que en nuestros experimentos encuentre buenas soluciones óptimas, pero llegue menos lejos en cobertura que otras alternativas óptimas.

seq-opt-bjolp
BJOLP (Big Joint Optimal Landmarks Planner) es un planificador óptimo basado en landmarks y en una combinación admisible de sus costes; en la literatura se describe como un enfoque conjunto de landmarks óptimos, y las notas recientes de Fast Downward lo relacionan con landmark_cost_partitioning. Es natural, por tanto, que pueda ofrecer una heurística muy informativa, aunque con mayor coste computacional por estado que enfoques más simples.

seq-opt-fdss-2
seq-opt-fdss-2 es la versión óptima de Fast Downward Stone Soup 2, es decir, otra configuración portfolio. Su comportamiento depende mucho de que alguna de sus configuraciones internas encaje bien con el dominio y de cuánto tiempo tenga disponible. Por eso, si funciona bien en nuestros experimentos, no debe leerse como “una heurística mejor”, sino como el efecto de una cartera de métodos óptimos competitivos.

Discusión de resultados óptimos

La diferencia respecto a los planificadores satisficing es muy clara. Los métodos óptimos resuelven instancias bastante más pequeñas, lo cual es exactamente lo esperable: no basta con encontrar una solución, sino que deben demostrar que no existe otra de menor coste.

Entre los tres planificadores analizados, los mejores resultados corresponden a:

- seq-opt-bjolp
- seq-opt-fdss-2

Ambos alcanzan tamaño 7 y obtienen el mismo coste óptimo (524), aunque seq-opt-fdss-2 lo hace en menos tiempo total. Por tanto, dentro de este conjunto experimental, seq-opt-fdss-2 ofrece el mejor compromiso entre cobertura y tiempo entre los planificadores óptimos.

Por su parte, seq-opt-lmcut se queda en tamaño 6. Aunque LM-Cut es una heurística admisible muy potente desde el punto de vista teórico, en este dominio parece sufrir más la combinación entre coste de cálculo heurístico y crecimiento del espacio de búsqueda. De nuevo aparece una idea importante: una heurística más informada no garantiza automáticamente un mejor rendimiento práctico si su cálculo también es más costoso.

2.2.8. Comparación entre satisficing y óptimos

La comparación entre ambas familias de planificadores es especialmente ilustrativa en esta parte, porque ahora la calidad de una solución ya no depende únicamente del número de acciones, sino de su coste total.

Los resultados muestran dos comportamientos muy distintos:

- Los planificadores satisficing, y en particular lama-first, alcanzan tamaños muy grandes en poco tiempo, pero sin garantía de optimalidad.
- Los planificadores óptimos obtienen planes de mucho menor coste, pero su cobertura se reduce drásticamente.

Esta diferencia puede observarse incluso en tamaños pequeños. Por ejemplo, en instancias como n=5 o n=6, los planificadores óptimos producen costes muy inferiores a los de los satisficing, lo que indica que el problema de optimización realmente importa: no basta con entregar los contenidos, sino que la forma concreta de hacerlo cambia mucho el coste final.

También se aprecia que coste mínimo y longitud mínima no son lo mismo. En un dominio con vuelos caros y acciones locales baratas, puede resultar preferible aceptar algunos pasos extra si eso evita desplazamientos costosos. Por tanto, un plan más largo no tiene por qué ser peor si su coste acumulado es menor.

Esta observación es central en la Parte 2.2, porque justifica precisamente el paso desde una planificación clásica no optimizada a una planificación orientada a coste.

2.2.9. Interpretación de los resultados

El comportamiento observado en los experimentos es coherente con la estructura del dominio.

En este problema, el coste dominante proviene de los desplazamientos entre localizaciones, mientras que las operaciones locales tienen coste bajo. Como consecuencia, un planificador eficaz debe encontrar no solo una forma de satisfacer todas las metas, sino una que reduzca desplazamientos caros o los aproveche mejor mediante el uso del transportador.

Esto hace que el transportador, que en 2.1 introducía complejidad adicional desde el punto de vista del espacio de búsqueda, pase ahora a tener también una dimensión claramente económica. No se trata solo de "cómo" mover cajas, sino de hacerlo de forma que compense realmente el coste de los trayectos.

En este contexto, los planificadores satisficing se benefician de su capacidad para hallar rápidamente soluciones buenas, mientras que los óptimos deben explorar mucho más para certificar que no existe una alternativa de menor coste. Esa exigencia adicional explica la gran diferencia de cobertura entre unos y otros.

El resultado especialmente destacado de toda esta parte es el de LAMA-first, que resuelve instancias muy superiores en tamaño al resto de satisficing y muchísimo mayores que las alcanzadas por los óptimos. En el extremo contrario, los planificadores óptimos muestran claramente el precio de la optimalidad: planes de mejor calidad, pero una frontera de resolución mucho más limitada.

2.2.10. Conclusiones de la Parte 2.2

La introducción de costes transforma de forma importante el problema de planificación. En esta nueva versión ya no basta con encontrar una solución, sino que interesa minimizar el coste total del plan. Esto hace que la comparación entre planificadores sea especialmente rica, porque permite analizar simultáneamente cobertura, tiempo de resolución y calidad de la solución obtenida.

Desde el punto de vista del modelado, la ampliación realizada resulta adecuada. El uso de :action-costs, la función total-cost, la función fly-cost y la métrica de minimización permiten expresar correctamente el objetivo de optimización dentro de las restricciones del formalismo PDDL empleado en la práctica.

Desde el punto de vista experimental, los resultados son muy claros:

- LAMA-first es, con diferencia, el planificador satisficing más potente en este dominio con costes.
- Metric-FF funciona bien en instancias pequeñas, pero su evaluación queda limitada por un error interno de optimización a partir de cierto tamaño.
- Los aliases satisficing seq-sat-fdss-2 y seq-sat-fd-autotune-2 obtienen una cobertura muy reducida en comparación con LAMA-first.
- Entre los óptimos, seq-opt-bjolp y seq-opt-fdss-2 son los más competitivos, alcanzando ambos tamaño 7.
- En todos los casos se confirma que la garantía de optimalidad reduce fuertemente la escalabilidad.

En conjunto, la Parte 2.2 muestra de forma muy clara la diferencia entre encontrar una buena solución rápido y garantizar la mejor solución posible. Esa diferencia, que ya aparecía implícitamente en la Parte 1, queda aquí plenamente explicitada gracias al uso de una función de coste y una métrica de optimización.

2.2.11. Resumen final de resultados

Planificadores satisficing

Planificador	Mayor tamaño resuelto	Coste	Longitud	Tiempo total (s)
Metric-FF	10*	2081	41	0.00
lama-first	97	33638	414	38.20
seq-sat-fdss-2	6	1677	23	0.0036
seq-sat-fd-autotune-2	5	1769	23	17.47

* A partir de tamaño 11 Metric-FF devolvió un error del planificador relacionado con la optimización, por lo que no se interpretó como timeout ni como prueba de no resolubilidad.

Planificadores óptimos

Planificador	Mayor tamaño resuelto	Coste óptimo	Longitud	Tiempo total (s)
seq-opt-lmcut	6	610	29	29.32
seq-opt-bjolp	7	524	33	7.40
seq-opt-fdss-2	7	524	33	3.79

3. Parte 3: Planificación con tiempos

3.1. Objetivo de la parte

En esta tercera parte de la práctica se amplía el dominio desarrollado en las Partes 1 y 2 para introducir planificación temporal. El objetivo ya no es minimizar el coste acumulado de las acciones, como en la Parte 2.2, sino modelar correctamente acciones durativas y permitir que el planificador explote concurrencia válida entre recursos cuando esta sea posible.

Para ello se ha utilizado el planificador OPTIC, que soporta acciones durativas y optimización temporal. La idea central de esta parte es representar el problema de forma que los tiempos de ejecución de las acciones influyan directamente en la calidad del plan, medida mediante el makespan o duración total del mismo.

Siguiendo el enunciado, se ha realizado lo siguiente:

- conversión del dominio a acciones durativas;
- asignación de duración variable a los vuelos, igual a fly-cost;
- asignación de duración fija 5 al resto de acciones;
- introducción de restricciones que impiden concurrencia inválida entre drones, transportadores y personas;
- ejecución de un benchmark con configuraciones desde 1 dron y 1 transportador hasta 5 drones y 5 transportadores;
- almacenamiento, para cada problema procesado, de la primera y la última solución devueltas por OPTIC dentro del intervalo de 1 minuto;
- comparación entre ambas soluciones en número de pasos y makespan.

La interpretación metodológica adoptada para esta parte ha sido la siguiente: dado que OPTIC es un planificador anytime, lo relevante no es si termina limpiamente antes del minuto, sino si es capaz de devolver una o más soluciones válidas dentro de ese intervalo. Por ello, en esta memoria se considera el dato de primera solución y última solución encontradas antes del timeout.

3.2. Adaptación del dominio a planificación temporal

3.2.1. Paso a acciones durativas

El dominio anterior se transformó a PDDL temporal sustituyendo las acciones instantáneas por acciones durativas. El conjunto de acciones se mantiene conceptualmente igual que en la Parte 2.1 y 2.2:

- fly
- pick-up
- deliver
- put-in-transporter
- move-transporter
- take-out-transporter

La diferencia es que ahora cada una de ellas ocupa un intervalo temporal. De este modo, el planificador puede decidir no solo qué acciones ejecutar, sino también cuándo iniciarlas y cuáles pueden solaparse sin conflicto.

3.2.2. Política de duraciones

Se ha adoptado una política de duraciones sencilla y coherente con el enunciado.

Las acciones de desplazamiento entre localizaciones usan como duración el valor de la función fly-cost. Esto se aplica a:

- fly
- move-transporter

Por el contrario, las acciones locales de manipulación tienen duración constante igual a 5:

- pick-up
- deliver
- put-in-transporter
- take-out-transporter

Esta decisión permite que el tiempo total del plan refleje de forma natural la estructura logística del problema. Los desplazamientos largos siguen siendo el componente dominante, mientras que las operaciones de carga, descarga y entrega tienen un coste temporal pequeño pero no despreciable.

3.2.3. Restricciones de concurrencia

Uno de los aspectos más importantes de esta parte ha sido impedir concurrencia inválida sin bloquear el paralelismo útil. Para ello se han introducido tres predicados de control de recurso:

- drone-free
- transporter-free
- person-free

La idea es la siguiente:

- un dron no puede ejecutar dos acciones incompatibles al mismo tiempo;
- un transportador no puede ser cargado, descargado o movido simultáneamente por varias acciones incompatibles;
- una misma persona no puede participar en dos entregas solapadas.

En la práctica, las acciones que utilizan alguno de estos recursos comprueban al inicio que el recurso está libre, lo bloquean al comenzar y lo liberan al terminar. Esto permite concurrencia real cuando las acciones afectan a recursos distintos, pero evita interferencias inválidas.

Aunque no se ha introducido un predicado específico del tipo crate-free, la exclusión sobre una misma caja queda garantizada por la propia evolución del estado. En particular, una caja situada en una localización deja de satisfacer at-crate en cuanto un dron inicia pick-up, y una caja almacenada en un transportador deja de satisfacer in-transporter en cuanto se inicia take-out-transporter. De forma análoga, una caja que está siendo sostenida por un dron ya no puede ser utilizada simultáneamente por otro, porque las acciones que la requieren comparten precondiciones incompatibles. Por tanto, el dominio también impide interferencias inválidas sobre una misma caja.

3.2.4. Comentario sobre el modelado

El comportamiento esencial del dominio sigue siendo el mismo que en 2.1 y 2.2. El dron puede transportar una caja directamente con su mano o agrupar varias cajas en un transportador y desplazarlo entre localizaciones. La diferencia fundamental es que ahora la calidad del plan ya no depende solo del número de pasos, sino también de cómo se ordenan y solapan temporalmente esas acciones.

Esta observación es importante porque, en planificación temporal, un plan con más acciones puede ser mejor que otro con menos acciones si consigue reducir el makespan total. Como se verá más adelante, este comportamiento aparece de forma clara en los resultados.

3.3. Adaptación del generador de problemas

El generador de problemas también se adaptó al nuevo dominio temporal.

En concreto:

- se mantienen las localizaciones aleatorias y el cálculo de fly-cost entre cada par de localizaciones;
- se eliminan los elementos ligados a total-cost, ya que en esta parte no se optimiza coste acumulado;
- se inicializan los predicados drone-free, transporter-free y person-free;
- se mantiene la inicialización de free-slots y de la cadena numérica siguiente para la capacidad de los transportadores;
- se añade la métrica (:metric minimize (total-time));
- se fija una semilla para que los problemas sean reproducibles y comparables entre configuraciones.

La semilla fija resulta especialmente importante en esta parte. Gracias a ella, para un mismo tamaño n las distintas configuraciones de drones y transportadores se evalúan sobre instancias comparables. Esto mejora la validez experimental del benchmark y evita que las diferenci1as entre configuraciones puedan atribuirse simplemente a variaciones aleatorias entre problemas.

3.4. Validación inicial

Antes de lanzar el benchmark completo se realizaron varias comprobaciones manuales para verificar que el dominio temporal era correcto.

Para validar la concurrencia del dominio temporal se ejecutó Optic sobre una instancia pequeña con 2 drones y 2 transportadores. En el plan obtenido se observa concurrencia real desde el instante 0.000, ya que dos acciones pick-up comienzan simultáneamente sobre drones distintos. A continuación, también aparecen dos acciones fly iniciadas al mismo tiempo. Esto confirma que el dominio permite paralelismo cuando no existe interferencia entre recursos. Además, las entregas a una misma persona no se solapan temporalmente, lo que es coherente con el uso del predicado person-free para evitar interacciones inválidas.

```
(env) javid@javi:~/PL1/PECL1_Criado-Dutu-Yanev/Parte3$ timeout 60s planutils run optic domain.pddl problem_d2_r2_l2_p2_c2_g2.pddl
Number of literals: 44
Constructing lookup tables: [10%] [20%] [30%] [40%] [50%] [60%] [70%] [80%] [90%] [100%]
Post filtering unreachable actions:  [10%] [20%] [30%] [40%] [50%] [60%] [70%] [80%] [90%] [100%]
Seeing if metric is defined in terms of task vars or a minimal value of makespan
- Yes it is
Recognised a monotonic-change-induced limit on -1.000*<special>
- Must be >=  the metric
Looking for achievers for goal index 0, fact (has-content person2 food) with fID 25
 (deliver drone1 crate1 person2 loc2 food) (deliver drone2 crate1 person2 loc2 food)
For limits: literal goal index 0, fact (has-content person2 food), could be achieved by operator (deliver drone1 crate1 person2 loc2 f
n-free person2) )
Looking for achievers for goal index 1, fact (has-content person2 medicine) with fID 27
 (deliver drone1 crate2 person2 loc2 medicine) (deliver drone2 crate2 person2 loc2 medicine)
For limits: literal goal index 1, fact (has-content person2 medicine), could be achieved by operator (deliver drone1 crate2 person2 lo
n (person-free person2) )
22% of the ground temporal actions in this problem are compression-safe
Initial heuristic = 7.000, admissible cost estimate 73.002
b (5.000 | 73.001)b (4.000 | 73.001)b (3.000 | 78.002)b (2.000 | 78.002)b (1.000 | 83.003)(G)
; LP calculated the cost

; Plan found with metric 83.003
; Theoretical reachable cost 83.004
; States evaluated so far: 21
; States pruned based on pre-heuristic cost lower bound: 0
; Time 0.10
0.000: (pick-up drone1 crate2 depot)  [5.000]
0.000: (pick-up drone2 crate1 depot)  [5.000]
5.001: (fly drone1 depot loc2)  [68.000]
5.001: (fly drone2 depot loc2)  [68.000]
73.002: (deliver drone1 crate2 person2 loc2 medicine)  [5.000]
78.003: (deliver drone2 crate1 person2 loc2 food)  [5.000]

 * All goal deadlines now no later than 83.003

Resorting to best-first search
Running WA* with W = 5.000, not restarting with goal states
b (6.000 | 73.001)b (5.000 | 73.001)b (4.000 | 73.001)(env) javid@javi:~/PL1/PECL1_Criado-Dutu-Yanev/Parte3$
```

También se realizó una validación manual del uso del transportador en una instancia pequeña con 1 dron y 1 transportador. En el plan temporal obtenido aparecen explícitamente las acciones put-in-transporter, move-transporter y take-out-transporter, lo que confirma que el modelado del transportador seguía siendo operativo tras la conversión temporal. En este caso no aparece concurrencia, lo cual es esperable al disponer de un único dron.

```
; Plan found with metric 108.008
; Theoretical reachable cost 108.009
; States evaluated so far: 152
; States pruned based on pre-heuristic cost lower bound: 0
; Time 0.17
0.000: (pick-up drone1 crate1 depot)  [5.000]
5.001: (put-in-transporter drone1 crate1 transporter1 depot n3 n4)  [5.000]
10.002: (pick-up drone1 crate2 depot)  [5.000]
15.003: (put-in-transporter drone1 crate2 transporter1 depot n2 n3)  [5.000]
20.004: (move-transporter drone1 transporter1 depot loc2)  [68.000]
88.005: (take-out-transporter drone1 crate2 transporter1 loc2 n2 n3)  [5.000]
93.006: (deliver drone1 crate2 person2 loc2 medicine)  [5.000]
98.007: (take-out-transporter drone1 crate1 transporter1 loc2 n3 n4)  [5.000]
103.008: (deliver drone1 crate1 person2 loc2 food)  [5.000]
```

3.5. Metodología experimental

3.5.1. Planificador utilizado y criterio de éxito

Para esta parte se ha utilizado el planificador temporal OPTIC. El criterio experimental adoptado ha sido el siguiente: una instancia se considera resuelta si OPTIC devuelve al menos una solución dentro de un tiempo máximo de 60 segundos.

Este criterio es importante porque OPTIC funciona en modo anytime. En consecuencia, no resulta especialmente relevante si el proceso termina limpiamente antes del minuto o si continúa buscando mejoras hasta agotar el tiempo disponible. Lo relevante para el benchmark es si el planificador es capaz de producir al menos un plan válido dentro del límite establecido.

Además, para cada problema procesado se registraron:

- el número total de soluciones encontradas;
- la primera solución devuelta por OPTIC;
- la última solución devuelta dentro del límite temporal;
- el tiempo de aparición de ambas;
- el makespan de ambas;
- y el número de pasos de ambas.

3.5.2. Configuraciones estudiadas

Se han estudiado las cinco configuraciones pedidas en el enunciado:

- 1 dron / 1 transportador
- 2 drones / 2 transportadores
- 3 drones / 3 transportadores
- 4 drones / 4 transportadores
- 5 drones / 5 transportadores

En todos los casos se ha mantenido la misma forma de instancia:

problem_dD_rR_lN_pN_cN_gN.pddl

con D = R y con N como medida global del tamaño del problema.

3.5.3. Localización del umbral de resolución

Para ahorrar tiempo experimental se empleó un barrido híbrido. En lugar de crecer siempre de 1 en 1 desde tamaños pequeños, se hizo primero un barrido grueso en saltos amplios, después un retroceso intermedio y, finalmente, un refinamiento de 1 en 1 en torno a la frontera de resolución.

Este procedimiento permite localizar con mucha más rapidez el mayor tamaño resoluble sin perder precisión en el tramo realmente importante. La consecuencia es que en el CSV detallado aparecen algunos tamaños no consecutivos, ya que se conservan todas las instancias que fueron realmente procesadas durante el barrido.

Por tanto, la tabla detallada no representa una sucesión completa de todos los tamaños, sino el conjunto exacto de problemas evaluados por el benchmark definitivo.

3.6. Resultados

3.6.1. Resumen por configuración

La siguiente tabla resume, para cada configuración, el mayor tamaño n para el que OPTIC fue capaz de devolver al menos una solución dentro de 60 segundos, así como el primer tamaño sin solución dentro de ese límite.

Configuración	**Máximo n con alguna solución en 60 s**	**Primer n sin solución en 60 s**
1 dron(es) / 1 transportador(es)	40	41
2 dron(es) / 2 transportador(es)	17	18
3 dron(es) / 3 transportador(es)	9	10
4 dron(es) / 4 transportador(es)	6	7
5 dron(es) / 5 transportador(es)	3	4

La primera observación es muy clara: la configuración 1/1 es, con diferencia, la que más lejos llega. A partir de ahí, el umbral de resolución cae de forma rápida conforme aumenta el número de drones y transportadores.

3.6.2. Comparación entre primera y última solución

A continuación se recoge, con una fila por problema procesado, la comparación pedida entre primera y última solución.

Configuración 1/1

n	Soluciones	t primera (s)	Makespan primera	Pasos primera	t última (s)	Makespan última	Pasos última
2	2	0.06	224.006	7	0.15	108.008	9
12	1	0.62	4371.05	51	0.62	4371.05	51
22	1	5.92	7359.096	97	5.92	7359.096	97
32	1	25.06	11765.143	144	25.06	11765.143	144
37	1	48.95	15444.165	166	48.95	15444.165	166
38	1	43.68	16002.169	170	43.68	16002.169	170
39	1	55.51	15368.175	176	55.51	15368.175	176
40	1	49.29	14685.172	173	49.29	14685.172	173
41	0	—	—	—	—	—	—
42	0	—	—	—	—	—	—

Configuración 2/2

n	Soluciones	t primera (s)	Makespan primera	Pasos primera	t última (s)	Makespan última	Pasos última
2	1	0.06	83.003	6	0.06	83.003	6
12	1	12.17	3088.037	65	12.17	3088.037	65
17	1	52.77	3488.049	88	52.77	3488.049	88
18	0	—	—	—	—	—	—
22	0	—	—	—	—	—	—

Configuración 3/3

n	Soluciones	t primera (s)	Makespan primera	Pasos primera	t última (s)	Makespan última	Pasos última
2	1	0.11	83.003	7	0.11	83.003	7
7	1	6.82	1039.012	33	6.82	1039.012	33
8	1	10.69	1019.014	38	10.69	1019.014	38
9	1	20.1	1216.014	43	20.1	1216.014	43
10	0	—	—	—	—	—	—
12	0	—	—	—	—	—	—

Configuración 4/4

n	Soluciones	t primera (s)	Makespan primera	Pasos primera	t última (s)	Makespan última	Pasos última
2	1	0.33	83.003	8	0.33	83.003	8
6	1	50.91	617.007	32	50.91	617.007	32
7	0	—	—	—	—	—	—
12	0	—	—	—	—	—	—

Configuración 5/5

n	Soluciones	t primera (s)	Makespan primera	Pasos primera	t última (s)	Makespan última	Pasos última
2	1	1.4	83.003	9	1.4	83.003	9
3	1	8.11	214.004	17	8.11	214.004	17
4	0	—	—	—	—	—	—
5	0	—	—	—	—	—	—
6	0	—	—	—	—	—	—
7	0	—	—	—	—	—	—
12	0	—	—	—	—	—	—

3.7. Interpretación de resultados

3.7.1. Diferencia entre primera y última solución

La configuración 1 dron / 1 transportador es la que mejor ilustra el comportamiento anytime de OPTIC.

En n = 2, la primera solución aparece prácticamente de inmediato, en 0.06 s, con makespan 224.006 y 7 pasos. Sin embargo, en 0.15 s ya aparece una solución mejor con makespan 108.008, aunque con 9 pasos. Es decir, el plan mejora mucho en tiempo total, pero empeora en longitud.

Este caso confirma una idea importante de la planificación temporal: un plan mejor no tiene por qué ser un plan más corto. En este dominio, el planificador puede introducir acciones adicionales de carga y descarga del transportador si con ello evita vuelos largos repetidos y reduce de forma notable el makespan.

Ahora bien, fuera de este caso pequeño, el comportamiento anytime apenas se aprecia en los datos finales. En la inmensa mayoría de las instancias con solución, OPTIC solo devuelve una solución dentro del minuto, de modo que primera y última solución coinciden. Esto no contradice el funcionamiento del planificador, sino que indica que, en este dominio y con este límite temporal, normalmente dispone de tiempo para encontrar un primer plan pero no para mejorarlo varias veces.

3.7.2. Evolución del makespan y del número de pasos

El makespan crece con rapidez al aumentar el tamaño del problema, como cabía esperar. En la configuración 1/1 pasa de 108.008 en n = 2 a más de 14000 en la frontera de resolución. En 2/2, el último problema con solución, n = 17, ya presenta makespan 3488.049. En 3/3, 4/4 y 5/5, aunque la frontera aparece antes, también se observa un crecimiento claro del tiempo total del plan.

El número de pasos también aumenta con n, aunque no de forma estrictamente monótona. Esto es razonable, porque cada valor de n corresponde a una instancia distinta generada aleatoriamente y no a una simple ampliación incremental de la anterior. Por ello, pequeñas oscilaciones en makespan o en longitud del plan no deben interpretarse como incoherencias del benchmark.

También resulta interesante observar que, para un mismo tamaño pequeño, disponer de más recursos sí puede reducir claramente el makespan. Por ejemplo, en n = 2, la configuración 1/1 acaba con makespan 108.008, mientras que las configuraciones con más recursos alcanzan 83.003. Esto indica que el paralelismo adicional sí puede mejorar la calidad temporal del plan cuando el problema todavía es lo bastante pequeño como para que OPTIC lo explote.

3.7.3. Papel de la concurrencia

La concurrencia válida es uno de los objetivos centrales de esta parte y ha quedado confirmada por las validaciones manuales.

En problemas pequeños con varios drones aparecen acciones simultáneas sobre recursos distintos, lo que demuestra que el dominio no fuerza una serialización artificial. Al mismo tiempo, las restricciones drone-free, transporter-free y person-free impiden solapamientos indebidos. En particular, las entregas dirigidas a una misma persona no aparecen superpuestas, lo que respalda el modelado de exclusión mutua sobre personas.

Desde el punto de vista experimental, esta concurrencia explica por qué configuraciones con más drones pueden producir planes de menor makespan en tamaños pequeños. El paralelismo es real y útil. El problema no está en que el dominio impida concurrencia válida, sino en que el planificador tiene que explorar un espacio de búsqueda mucho mayor para aprovecharla.

3.7.4. Papel del transportador

El transportador sigue desempeñando un papel importante en la Parte 3. Las pruebas manuales muestran que sus acciones siguen apareciendo en planes válidos del dominio temporal y, además, el caso 1/1, n = 2 evidencia muy bien su utilidad: la mejora entre la primera y la última solución se explica precisamente por una reorganización del plan que utiliza mejor el transportador y reduce desplazamientos largos repetidos.

Esto encaja con la intuición del problema. Aunque las acciones de meter y sacar cajas añaden pasos, pueden compensar sobradamente cuando permiten agrupar entregas y reducir vuelos costosos en tiempo. Por tanto, el dominio temporal conserva correctamente la motivación introducida en la Parte 2: el transportador no sirve para acortar necesariamente el plan en número de acciones, sino para hacerlo más eficiente desde el punto de vista logístico.

3.7.5. Por qué más recursos no siempre ayudan a OPTIC

El resultado más llamativo de esta parte es que añadir más drones y más transportadores no mejora la escalabilidad del planificador; de hecho, la empeora claramente.

La explicación no es que más recursos produzcan peores planes, sino que hacen mucho más difícil la búsqueda. Con más drones y transportadores aumentan:

- las posibles asignaciones entre drones, cajas, personas y transportadores;
- las simetrías entre recursos equivalentes;
- las combinaciones temporales posibles;
- y, en general, el espacio de búsqueda que OPTIC debe explorar.

Por ello, aunque disponer de más recursos puede permitir mejores planes en principio, el coste de encontrar esos planes crece muy deprisa. Esto se observa con claridad en la frontera experimental:

- 1/1 resuelve hasta n = 40;
- 2/2 hasta n = 17;
- 3/3 hasta n = 9;
- 4/4 hasta n = 6;
- 5/5 hasta n = 3.

Se trata, por tanto, de un caso muy claro en el que mejorar el poder expresivo del dominio y aumentar los recursos disponibles no simplifica necesariamente la búsqueda para un planificador independiente del dominio.

3.7.6. Sobre el uso del timeout

Todos los casos con solución registrados en el benchmark aparecen como timeout_con_solucion. Esto significa que OPTIC encontró al menos una solución dentro del minuto, pero continuó ejecutándose hasta agotar el límite temporal. Este comportamiento es totalmente coherente con su naturaleza anytime.

Por ello, en la memoria conviene formular con cuidado la conclusión experimental. Lo correcto no es afirmar que OPTIC “termina de resolver” el problema en menos de 60 segundos, sino que “es capaz de devolver al menos una solución dentro de 60 segundos”. Ese es precisamente el criterio relevante en esta parte.

3.8. Conclusiones

La tercera parte de la práctica ha permitido extender el dominio anterior a un contexto de planificación temporal con acciones durativas y concurrencia controlada.

En primer lugar, el dominio temporal modela correctamente el problema planteado. Las acciones de desplazamiento utilizan duraciones dependientes de fly-cost, mientras que las acciones locales mantienen duración constante. Esta combinación resulta natural y hace que la métrica temporal tenga un efecto real sobre el comportamiento del planificador.

En segundo lugar, la introducción de drone-free, transporter-free y person-free ha permitido representar exclusión mutua entre recursos sin impedir concurrencia válida. Las validaciones manuales muestran que OPTIC puede ejecutar acciones en paralelo cuando afectan a recursos distintos, pero evita solapamientos incorrectos.

En tercer lugar, los resultados experimentales muestran que OPTIC explota bien el paralelismo en instancias pequeñas, pero sufre una caída rápida de escalabilidad cuando aumenta el número de drones y transportadores. Esto no implica que más recursos generen peores planes, sino que multiplican las opciones de búsqueda y dificultan mucho la planificación temporal.

En cuarto lugar, la comparación entre primera y última solución confirma que, en planificación temporal, la calidad del plan no debe medirse únicamente por el número de pasos. El caso 1/1, n = 2 muestra de forma muy clara que un plan con más acciones puede ser mucho mejor si reduce el makespan.

En conjunto, la Parte 3 no solo valida el modelado temporal del dominio, sino que también muestra de forma clara la relación entre duración de acciones, concurrencia, uso del transportador y dificultad práctica de búsqueda en planificación temporal.

ANEXO I. Comandos a utilizar

1.1. Ejecución del dominio y los problemas

Para comprobar que el dominio emergency-services y los dos problemas hechos a mano están bien definidos, basta con guardar los archivos como domain.pddl, problem1.pddl y problem2.pddl en la misma carpeta, activar nuestro entorno virtual donde tengamos instalados los comandos que usaremos y ejecutar un planificador clásico sobre cada uno de ellos. La forma más sencilla de validarlos es con pyperplan:

```
pyperplan domain.pddl problem1.pddl

pyperplan domain.pddl problem2.pddl
```

Con estos comandos se obtiene un plan válido para cada problema y se puede verificar que las acciones fly, pick-up y deliver funcionan correctamente. Si se prefiere hacer una comprobación rápida con FF, también puede usarse:

```
planutils run ff domain.pddl problem1.pddl

planutils run ff domain.pddl problem2.pddl
```

1.2. Generación automática de problemas y benchmark con FF

Para estudiar hasta qué tamaño de problema resuelve FF en menos de un minuto, se utilizó el script benchmark2.py. Este script genera automáticamente instancias de tamaño creciente con generate-problem2.py, ejecuta FF sobre ellas y refina el último tramo cuando aparece el primer timeout.

El benchmark se lanza con:

```
python3 benchmark2.py
```

Este comando genera el fichero de resultados resultados.csv. A partir de ese CSV puede construirse la gráfica de tiempos mediante:

```
python3 grafica.py
```

De este modo se obtiene la figura utilizada en la memoria para representar la evolución del tiempo de resolución de FF según el tamaño del problema.

1.3. Comparativa con Pyperplan

Los experimentos de este apartado se separaron en tres scripts distintos.

Para comparar BFS, IDS, A* con hMAX y GBFS con hMAX en problemas de tamaño creciente, se utilizó:

```
python3 1_3_1.py
```

Este script genera o reutiliza instancias del tipo problem_d1_r0_lN_pN_cN_gN.pddl y registra el mayor tamaño resuelto por cada algoritmo dentro del límite de 60 segundos.

Para repetir el experimento equivalente a 1.3.2 sobre el problema fijo de tamaño 8, se utilizó:

```
python3 1_3_2.py
```

Este script debe ejecutarse teniendo disponible en la carpeta de trabajo el problema fijo:

problem_d1_r0_l8_p8_c8_g8.pddl

Finalmente, para el experimento equivalente a 1.3.3 sobre el problema fijo de tamaño 6, se utilizó:

```
python3 1_3_3.py
```

En este caso debe estar disponible el problema:

problem_d1_r0_l6_p6_c6_g6.pddl

Cada script genera su correspondiente fichero CSV con los resultados empleados en las tablas de la memoria.

2.1. Ejecución del dominio con transportadores

Para la Parte 2.1 se utilizó un nuevo dominio, emergency-services-transporters, junto con el generador generate-problem.py. No obstante, para los experimentos equivalentes a 1.3.2 y 1.3.3 no se usaron instancias aleatorias nuevas, sino versiones adaptadas de los problemas fijos de la Parte 1.

Una vez guardados el dominio domain.pddl y los problemas adaptados problem_d1_r1_l8_p8_c8_g8.pddl y problem_d1_r1_l6_p6_c6_g6.pddl, las pruebas se ejecutaron con:

```
python3 2_1_3_2.py

python3 2_1_3_3.py
```

El primer comando reproduce la comparación satisficing sobre la instancia de tamaño 8 y el segundo reproduce la comparación entre algoritmos óptimos sobre la instancia de tamaño 6.

Si se desea generar instancias adicionales del nuevo dominio con un transportador, pueden usarse comandos como los siguientes:

```
python3 generate-problem.py -d 1 -r 1 -l 8 -p 8 -c 8 -g 8

python3 generate-problem.py -d 1 -r 1 -l 6 -p 6 -c 6 -g 6
```

2.2. Ejecución del dominio con costes y optimización

Para la Parte 2.2 se utilizó el mismo dominio con transportadores, pero ampliado con :action-costs, junto con un generador de problemas que inicializa total-cost, define todos los valores de fly-cost y añade la métrica de minimización.

El benchmark completo se ejecutó con:

```
python3 benchmark.py
```

Este script prueba planificador por planificador, incrementando el tamaño del problema hasta localizar el mayor N resuelto dentro de 60 segundos. Los resultados se guardan en los CSV detallado y resumen utilizados posteriormente en la memoria.

Además del benchmark automático, los planificadores también pueden lanzarse manualmente sobre una instancia concreta. Por ejemplo, para un problema de tamaño 5:

```
planutils run metric-ff domain.pddl problem_d1_r1_l5_p5_c5_g5.pddl

planutils run lama-first -- domain.pddl problem_d1_r1_l5_p5_c5_g5.pddl

planutils run downward -- --alias seq-sat-fdss-2 --search-time-limit 60s domain.pddl problem_d1_r1_l5_p5_c5_g5.pddl

planutils run downward -- --alias seq-sat-fd-autotune-2 --search-time-limit 60s domain.pddl problem_d1_r1_l5_p5_c5_g5.pddl

planutils run downward -- --alias seq-opt-lmcut domain.pddl problem_d1_r1_l5_p5_c5_g5.pddl

planutils run downward -- --alias seq-opt-bjolp domain.pddl problem_d1_r1_l5_p5_c5_g5.pddl

planutils run downward -- --alias seq-opt-fdss-2 --search-time-limit 60s domain.pddl problem_d1_r1_l5_p5_c5_g5.pddl
```

Estos comandos permiten reproducir manualmente el comportamiento de cada planificador sobre una instancia concreta y comprobar tanto el tiempo como el coste de la solución.

2.3. Limpieza y repetición de experimentos

Al trabajar con Fast Downward y otros planificadores de optimización, es habitual que se generen ficheros auxiliares como sas_plan o output.sas. Para evitar que esos archivos interfieran en ejecuciones posteriores, conviene eliminarlos después de cada prueba manual con:

```
rm -f sas_plan*

rm -f output.sas
```

Si se desea repetir un experimento completo desde cero, lo recomendable es mantener en la misma carpeta:

- domain.pddl
- el script correspondiente (benchmark2.py, 1_3_1.py, 2_1_3_2.py, benchmark.py, etc.)
- y los problemas fijos necesarios en cada caso.

A partir de ahí, basta con volver a ejecutar el script del apartado correspondiente para regenerar los resultados y actualizar los CSV usados en la memoria.

3.1. Ejecución manual del dominio temporal con OPTIC

Para comprobar que el dominio temporal y los problemas de prueba están bien definidos, basta con guardar domain.pddl y el problema correspondiente en la misma carpeta y ejecutar OPTIC mediante planutils. Por ejemplo:

```
planutils run optic domain.pddl problem_d1_r1_l4_p4_c4_g4.pddl

planutils run optic domain.pddl problem_d2_r2_l2_p2_c2_g2.pddl
```

Si se desea forzar manualmente el mismo límite de tiempo que en el benchmark:

```
timeout 60s planutils run optic domain.pddl problem_d1_r1_l4_p4_c4_g4.pddl

timeout 60s planutils run optic domain.pddl problem_d2_r2_l2_p2_c2_g2.pddl
```

3.2. Generación automática de problemas temporales

El generador de la Parte 3 produce problemas compatibles con el dominio temporal. Algunos ejemplos de uso son los siguientes:

```
python3 generate-problem.py -d 1 -r 1 -l 4 -p 4 -c 4 -g 4

python3 generate-problem.py -d 2 -r 2 -l 2 -p 2 -c 2 -g 2

python3 generate-problem.py -d 3 -r 3 -l 6 -p 6 -c 6 -g 6
```

3.3. Benchmark temporal con OPTIC

Para reproducir el benchmark completo de la Parte 3, lo recomendable es dejar el script definitivo con el nombre benchmark.py dentro de la carpeta de esta parte. En ese caso, bastaría con ejecutar:

```
python3 benchmark.py
```

Si se quiere lanzar una configuración concreta:

```
python3 benchmark.py --drones 1 --carriers 1

python3 benchmark.py --drones 2 --carriers 2

python3 benchmark.py --drones 3 --carriers 3

python3 benchmark.py --drones 4 --carriers 4

python3 benchmark.py --drones 5 --carriers 5
```

Si se desea reiniciar desde cero una única configuración y volver a calcularla:

```
python3 benchmark.py --drones 3 --carriers 3 --reset-config
```

3.4. Limpieza y repetición de experimentos

Para repetir los experimentos desde cero sin conservar resultados anteriores:

```
rm -rf resultados_3

python3 benchmark.py
```

Si además se desea borrar los problemas generados automáticamente:

```
rm -f problem_d*_r*_l*_p*_c*_g*.pddl
```